DATABASE IMPLEMENTATION

Implementation on GCP

The database has been implemented on GCP for the project:

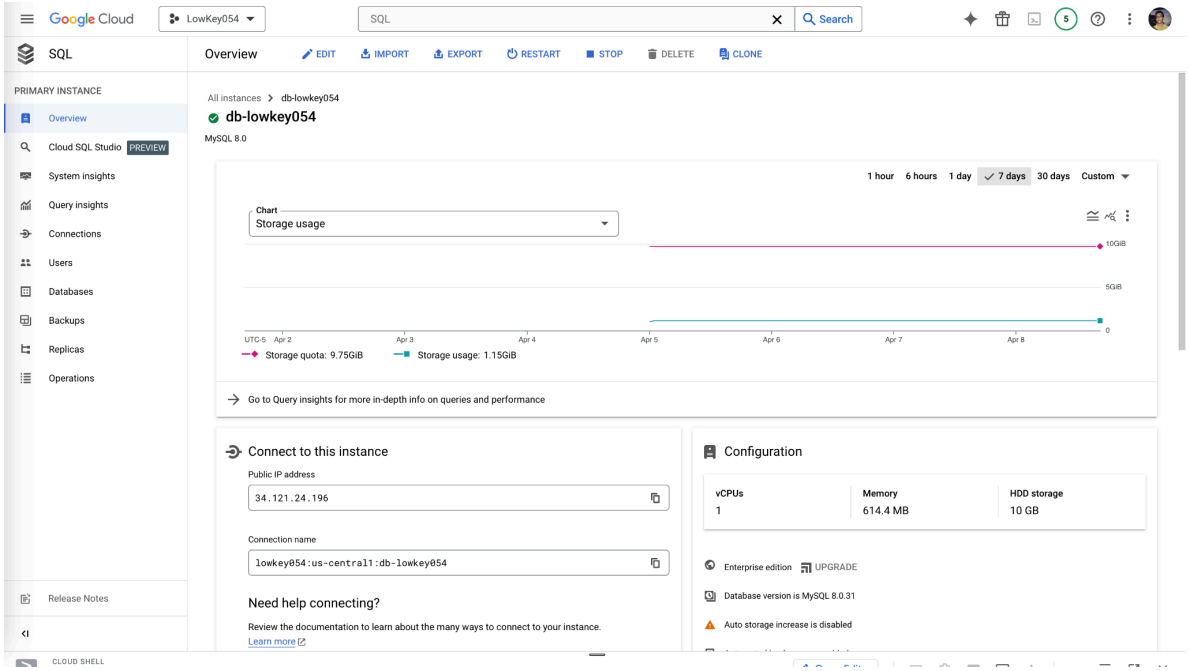

Fig: GCP Dashboard for the database

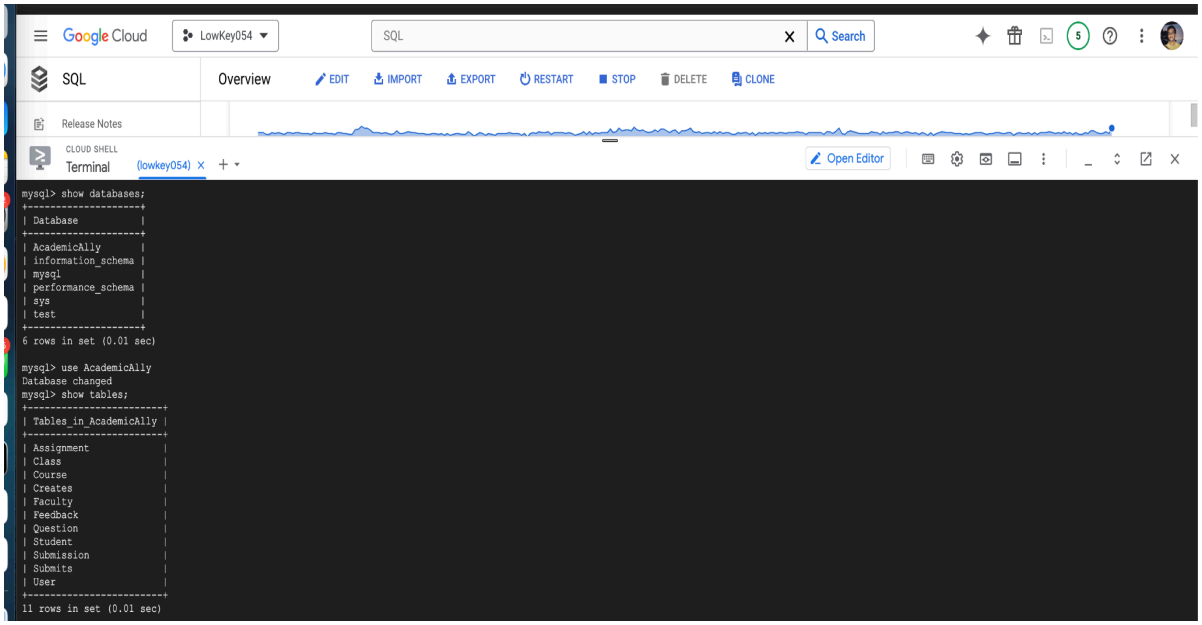

Fig: Output showing list of tables

DATABASE DESIGN

The current database design includes the creation of the following tables:

1. User
2. Class
3. Student
4. Faculty
5. Course
6. Assignment
7. Question
8. Submission
9. Submits
10. Creates
11. Feedback

DDL Commands:-

1)

```
CREATE TABLE User (
      UserID VARCHAR(20) NOT NULL,
      UserType VARCHAR(10) NOT NULL,
      Password VARCHAR(25) NOT NULL,
      PRIMARY KEY (UserID)
);
```

2)

```
CREATE TABLE Class (
      ClassID VARCHAR(20) NOT NULL,
      ClassName VARCHAR(20) NOT NULL,
      PRIMARY KEY (ClassID)
 );
```

3)

```
CREATE TABLE Student (
      StudentID VARCHAR(20) NOT NULL,
      FirstName VARCHAR(200) NOT NULL,
      LastName VARCHAR(200) NOT NULL,
      ClassID VARCHAR(20) NOT NULL,
      DoB DATE NOT NULL,
      PhoneNumber VARCHAR(10) NOT NULL,
      Email VARCHAR(30),
```

```
        EnrollmentDate DATE NOT NULL,
        PRIMARY KEY (StudentID),
        FOREIGN KEY (StudentID) REFERENCES User(UserID) ON DELETE
        CASCADE,
        FOREIGN KEY (ClassID) REFERENCES Class(ClassID) ON DELETE
        CASCADE
    );
```

4)

```
    CREATE TABLE Faculty (
        FacultyID VARCHAR(20) NOT NULL,
        FirstName VARCHAR(200) NOT NULL,
        LastName VARCHAR(200) NOT NULL,
        Phone VARCHAR(20) NOT NULL,
        Email VARCHAR(30) NOT NULL,
        DoB DATE,
        YearOfJoining DATE NOT NULL,
        PRIMARY KEY (FacultyID),
        FOREIGN KEY (FacultyID) REFERENCES User(UserID) ON DELETE
        CASCADE
    );
```

5)

```
    CREATE TABLE Course(
        CourseID VARCHAR(20) NOT NULL,
        FacultyID VARCHAR(20) NOT NULL,
        ClassID VARCHAR(20) NOT NULL,
        PRIMARY KEY (CourseID),
        FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID)
        ON DELETE CASCADE,
        FOREIGN KEY (ClassID) REFERENCES Class(ClassID) ON
        DELETE CASCADE
    );
```

6)

```
    CREATE TABLE Assignment (
        AssignmentID VARCHAR(20) NOT NULL,
        CourseID VARCHAR(20) NOT NULL,
        Description VARCHAR(8000),
        Deadline DATETIME NOT NULL,
        PRIMARY KEY(AssignmentID),
        FOREIGN KEY(CourseID) REFERENCES Course(CourseID) ON
        DELETE CASCADE
    );
```

7)

```
CREATE TABLE Question (
      QuestionID VARCHAR(20) NOT NULL,
      AssignmentID VARCHAR(20) NOT NULL,
      Type VARCHAR(3),
      Question VARCHAR(8000),
      Topic VARCHAR(100),
      CorrectAnswer VARCHAR(1) NOT NULL,
      AssumedDifficulty VARCHAR(1) NOT NULL,
      PRIMARY KEY (QuestionID),
      FOREIGN KEY (AssignmentID) REFERENCES
      Assignment(AssignmentID) ON DELETE CASCADE
 );
```

8)

```
CREATE TABLE Submission(
      SubmissionID VARCHAR(20) NOT NULL,
      AssignmentID VARCHAR(20) NOT NULL,
      FacultyID VARCHAR(20) NOT NULL,
      ObjScore INT,
      SubScore INT,
      TotalScore INT,
      SubjAnswer VARCHAR(8000),
      PRIMARY KEY (SubmissionID),
      FOREIGN KEY (AssignmentID) REFERENCES
      Assignment(AssignmentID) ON DELETE CASCADE,
      FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID) ON
      DELETE CASCADE
);
```

9)

```
CREATE TABLE Submits(
      SubmissionID VARCHAR(20) NOT NULL,
      StudentID VARCHAR(20) NOT NULL,
      PRIMARY KEY (SubmissionID,StudentID),
      FOREIGN KEY (SubmissionID) REFERENCES
      Submission(SubmissionID) ON DELETE CASCADE,
      FOREIGN KEY (StudentID) REFERENCES Student(StudentID) ON
      DELETE CASCADE
);
```

10)

```
CREATE TABLE Creates (
      AssignmentID VARCHAR(20) NOT NULL,
      FacultyID VARCHAR(20) NOT NULL,
```

```
        PRIMARY KEY (AssignmentID),
        FOREIGN KEY (AssignmentID) REFERENCES
        Assignment(AssignmentID) ON DELETE CASCADE,
        FOREIGN KEY (FacultyID) REFERENCES Faculty(FacultyID) ON
        DELETE CASCADE
    );
```

11)

```
    CREATE TABLE Feedback (
        StudentID VARCHAR(20) NOT NULL,
        AssignmentID VARCHAR(20) NOT NULL,
        Content VARCHAR(8000),
        PRIMARY KEY (StudentID, AssignmentID),
        FOREIGN KEY (AssignmentID) REFERENCES
        Assignment(AssignmentID) ON DELETE CASCADE,
        FOREIGN KEY (StudentID) REFERENCES Student(StudentID) ON
        DELETE CASCADE
    );
```

COUNT QUERIES

```
mysql>
mysql> SELECT COUNT(*) FROM Assignment;
+----------+
| COUNT(*) |
+----------+
|     1000 |
+----------+
1 row in set (0.13 sec)

mysql> SELECT COUNT(*) FROM Student;
+----------+
| COUNT(*) |
+----------+
|     1500 |
+----------+
1 row in set (0.14 sec)

mysql> SELECT COUNT(*) FROM Question;
+----------+
| COUNT(*) |
+----------+
|    10377 |
+----------+
1 row in set (0.25 sec)
```

Fig: Row count of tables

The tables Assignment, Student and Question have 1000, 1500 and 10377 entries respectively.

ADVANCED QUERIES

1) QUERY 1

Use Case - A Faculty wants to find the average difficulty of all assignments with objective questions for class 9D.

```
SELECT
  FacultyID, CourseID, AssignmentID,
  ( SUM((AssumedDifficulty = 'E') * 1.0 * cnt) +
    SUM((AssumedDifficulty = 'M') * 2.0 * cnt) +
    SUM((AssumedDifficulty = 'H') * 3.0 * cnt)
  )/SUM(cnt) AS AssignmentDifficulty
FROM
  (
    SELECT
      FacultyID, CourseID, AssignmentID, AssumedDifficulty, COUNT(*) AS
cnt
    FROM
      Faculty NATURAL JOIN Course NATURAL JOIN Assignment NATURAL JOIN
Question
    WHERE
      AssignmentID LIKE '%_9D_%' AND Type='O'
    GROUP BY
      FacultyID, CourseID,AssignmentID, AssumedDifficulty
  ) AS
SubQuery
GROUP BY
  FacultyID, CourseID, AssignmentID;
```

```
mysql> SELECT
    ->   FacultyID, CourseID, AssignmentID,
    ->   ( SUM((AssumedDifficulty = 'E') * 1.0 * cnt) +
    ->     SUM((AssumedDifficulty = 'M') * 2.0 * cnt) +
    ->     SUM((AssumedDifficulty = 'H') * 3.0 * cnt)
    ->   )/SUM(cnt) AS AssignmentDifficulty
    -> FROM
    ->   (
    ->     SELECT
nt  ->       FacultyID, CourseID, AssignmentID, AssumedDifficulty, COUNT(*) AS c
    ->     FROM
estion       Faculty NATURAL JOIN Course NATURAL JOIN Assignment NATURAL JOIN Qu
    ->     WHERE
    ->       AssignmentID LIKE '%_9D_%' AND Type='O'
    ->     GROUP BY
    ->       FacultyID, CourseID,AssignmentID, AssumedDifficulty
    ->   ) AS
    -> SubQuery
    -> GROUP BY
    ->   FacultyID, CourseID, AssignmentID;
```

Fig: Advanced Query 1

FacultyID	CourseID	AssignmentID	AssignmentDifficulty
IU232618	MAT_9D	MAT_9D_HW1	2.13333
IU232618	MAT_9D	MAT_9D_HW2	1.80000
IU232618	MAT_9D	MAT_9D_HW3	1.86667
IU232618	MAT_9D	MAT_9D_HW4	2.00000
IU232618	MAT_9D	MAT_9D_HW5	1.73333
IU364794	GK_9D	GK_9D_HW1	2.50000
IU364794	GK_9D	GK_9D_HW2	1.83333
IU364794	GK_9D	GK_9D_HW3	1.83333
IU364794	GK_9D	GK_9D_HW4	2.33333
IU364794	GK_9D	GK_9D_HW5	1.83333
IU449901	ENG_9D	ENG_9D_HW1	1.50000
IU449901	ENG_9D	ENG_9D_HW2	2.00000
IU449901	ENG_9D	ENG_9D_HW3	2.00000
IU449901	ENG_9D	ENG_9D_HW4	1.50000
IU449901	ENG_9D	ENG_9D_HW5	2.00000

15 rows in set (0.01 sec)

Fig: First 15 rows of Query 1

2) QUERY 2

Use Case - A student would like to know all the assignments with upcoming deadlines that he has to submit.

```
(
SELECT
      AssignmentID, StudentID, deadline
FROM (
  SELECT
    ClassID, CourseID, AssignmentID, StudentID, deadline
  FROM
      Student NATURAL JOIN Class NATURAL JOIN Course NATURAL JOIN
Assignment
  WHERE
    ClassID = '10A' AND deadline < '2024-04-01' AND StudentID =
'SID20131143'
  GROUP BY
      StudentID, ClassID, CourseID, AssignmentID, deadline
) AS T1)
EXCEPT
SELECT
      AssignmentID, StudentID, deadline
FROM
      Submits NATURAL JOIN Submission NATURAL JOIN Assignment
WHERE
      Deadline < '2024-04-01' AND StudentID = 'SID20131143'
GROUP BY
      StudentID, AssignmentID, deadline;
```

```
mysql> (
    -> SELECT
    -> AssignmentID, StudentID, deadline
    -> FROM (
    ->   SELECT
    ->     ClassID, CourseID, AssignmentID, StudentID, deadline
    ->   FROM
    -> Student NATURAL JOIN Class NATURAL JOIN Course NATURAL JOIN Assignment
    ->   WHERE
143'->     ClassID = '10A' AND deadline < '2024-04-01' AND StudentID = 'SID20131
    ->   GROUP BY
    -> StudentID, ClassID, CourseID, AssignmentID, deadline
    -> ) AS T1)
    -> EXCEPT
    -> SELECT
    -> AssignmentID, StudentID, deadline
    -> FROM
    -> Submits NATURAL JOIN Submission NATURAL JOIN Assignment
    -> WHERE
    -> Deadline < '2024-04-01' AND StudentID = 'SID20131143'
    -> GROUP BY
    -> StudentID, AssignmentID, deadline;
```

Fig: Advanced Query 2

```
+--------------+-------------+---------------------+
| AssignmentID | StudentID   | deadline            |
+--------------+-------------+---------------------+
| ENG_10A_HW1  | SID20131143 | 2024-02-27 00:00:00 |
| GK_10A_HW1   | SID20131143 | 2024-02-27 00:00:00 |
| SCI_10A_HW1  | SID20131143 | 2024-02-27 00:00:00 |
+--------------+-------------+---------------------+
3 rows in set (0.10 sec)
```

Fig: Output is less than 15 rows.

3) QUERY 3

Use Case - A faculty would like to know the details of students who are below the class average for all the assignments in the courses taken by the faculty.

```
Select
      StudentID, email, avg_score, TotalScore, FirstName, LastName,
AssignmentID
from
Submits NATURAL JOIN (Select AssignmentID,
                        AVG(TotalScore) AS avg_score
                        from
```

```
                        Submission NATURAL JOIN Assignment
                        Group by AssignmentID) As T1
NATURAL JOIN Submission NATURAL JOIN Student
where TotalScore < avg_score;
```

```
mysql> Select
D   -> StudentID, email, avg_score, TotalScore, FirstName, LastName, AssignmentI
    -> from
    -> Submits NATURAL JOIN (Select AssignmentID,
    -> AVG(TotalScore) AS avg_score
    -> from
    -> Submission NATURAL JOIN Assignment
    -> Group by AssignmentID) As T1
    -> NATURAL JOIN Submission NATURAL JOIN Student
    -> where TotalScore < avg_score;
+-------------+-----------------------------+-----------+-----------+-----------+-
```

Fig: Advanced Query 3

StudentID	email	avg_score	TotalScore	FirstName	LastName	AssignmentID
SID20131231	rodney.hallwood@gmail.com	69.1000	67	Rodney	Hallwood	MAT_10A_HW1
SID20131384	alice.cooper@gmail.com	69.1000	45	Alice	Cooper	MAT_10A_HW1
SID20131420	isabelle.boatman@gmail.com	69.1000	34	Isabelle	Boatman	MAT_10A_HW1
SID20131529	clifford.dorsey@gmail.com	69.1000	69	Clifford	Dorsey	MAT_10A_HW1
SID20131582	stephen.middleton@gmail.com	69.1000	10	Stephen	Middleton	MAT_10A_HW1

5 rows in set (0.03 sec)

Fig: Output is less than 15 rows

4) QUERY 4

Use case - A faculty wants to know the average score of an assignment for one of his courses and the number of students who have submitted the assignment dependent on a deadline.

```
SELECT
      Course.FacultyID, Course.CourseID, Avg(Submission.TotalScore) AS
      AvgTotalScore, Count(DISTINCT Submits.StudentID)     AS
      NumberOfStudentsSubmitted, Deadline
FROM
      Course LEFT JOIN Assignment
              ON Course.CourseID = Assignment.CourseID
       LEFT JOIN Submission
              ON Assignment.AssignmentID = Submission.AssignmentID
       LEFT JOIN Submits
              ON Submission.SubmissionID = Submits.SubmissionID
WHERE
      Course.Facultyid = 'IU175125' AND Assignment.Deadline <
'2024-04-28'
GROUP BY
```

```
Course.FacultyID, Course.CourseID, Deadline;
```

```
mysql> SELECT
    -> Course.FacultyID, Course.CourseID, Avg(Submission.TotalScore) AS AvgTotalScore, Count(DISTINCT Submits.StudentID)     AS NumberOfStudentsSubmitted, Deadline
    -> FROM
    -> Course LEFT JOIN Assignment
    ->              ON Course.CourseID = Assignment.CourseID
    ->       LEFT JOIN Submission
    ->              ON Assignment.AssignmentID = Submission.AssignmentID
    ->       LEFT JOIN Submits
    ->              ON Submission.SubmissionID = Submits.SubmissionID
    -> WHERE
    -> Course.Facultyid = 'IU175125' AND Assignment.Deadline < '2024-04-28'
    -> GROUP BY
    -> Course.FacultyID, Course.CourseID, Deadline;
+-----------+----------+----------------+---------------------------+---------------------+
```

Fig: Advanced Query 4

FacultyID	CourseID	AvgTotalScore	NumberOfStudentsSubmitted	Deadline
IU175125	MAT_10A	69.1000	10	2024-02-27 00:00:00
IU175125	MAT_10A	98.0000	1	2024-04-27 00:00:00
IU175125	MAT_10B	41.0000	1	2024-02-27 00:00:00
IU175125	MAT_10B	NULL	0	2024-04-27 00:00:00
IU175125	MAT_10C	NULL	0	2024-02-27 00:00:00
IU175125	MAT_10C	NULL	0	2024-04-27 00:00:00
IU175125	MAT_10D	NULL	0	2024-02-27 00:00:00
IU175125	MAT_10D	NULL	0	2024-04-27 00:00:00
IU175125	MAT_10E	NULL	0	2024-02-27 00:00:00
IU175125	MAT_10E	NULL	0	2024-04-27 00:00:00

10 rows in set (0.00 sec)

Fig: Output is less than 15 rows

INDEXING ANALYSIS

STRATEGY: Our strategy for choosing columns for indexing is to go query by query. Since indexing is a database-wide implementation, for queries that are operating on tables that are not related to tables operated upon by other queries we have tried to maintain the minimal number of columns for indexing. For tables with common columns being queried, we have considered the effect of the common index on both queries.

A) QUERY 1

1) Original Explain Analyze:- without any added indices

```
mysql> SHOW INDEXES FROM Question;
+----------+------------+--------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table    | Non_unique | Key_name     | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----------+------------+--------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Question |          0 | PRIMARY      |            1 | QuestionID   | A         |        9547 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Question |          1 | AssignmentID |            1 | AssignmentID | A         |         993 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+----------+------------+--------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.01 sec)
```

Fig: Indexes on Questions Table

```
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------+
| -> Table scan on <temporary>  (actual time=2.473..2.476 rows=20 loops=1)
    -> Aggregate using temporary table  (actual time=2.472..2.472 rows=20 loops=1)
        -> Table scan on SubQuery  (cost=2.50..2.50 rows=0) (actual time=2.344..2.351 rows=50 loops=1)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=2.343..2.343 rows=50 loops=1)
                -> Table scan on <temporary>  (actual time=2.294..2.303 rows=50 loops=1)
                    -> Aggregate using temporary table  (actual time=2.292..2.292 rows=50 loops=1)
                        -> Nested loop inner join  (cost=909.29 rows=140) (actual time=0.331..1.961 rows=220 loops=1)
                            -> Nested loop inner join  (cost=860.24 rows=140) (actual time=0.323..1.840 rows=220 loops=1)
                                -> Nested loop inner join  (cost=291.10 rows=146) (actual time=0.274..1.135 rows=20 loops=1)
                                    -> Covering index scan on Course using faculty_idx  (cost=21.00 rows=200) (actual time=0.040..0.082 rows=200 loops=1)
                                    -> Filter: (Assignment.AssignmentID like '%_9D_%')  (cost=0.69 rows=1) (actual time=0.005..0.005 rows=0 loops=200)
                                        -> Covering index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=0.69 rows=7) (actual time=0.003..0.004 rows=5 loops=200)
                                -> Filter: (Question.`Type` = 'O')  (cost=2.94 rows=1) (actual time=0.014..0.032 rows=11 loops=20)
                                    -> Index lookup on Question using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=2.94 rows=10) (actual time=0.014..0.030 rows=11 loops=20)
                            -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)  (cost=0.25 rows=1) (actual time=0.000..0.000 rows=1 loops=220)
 |
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
```

Fig: Initial Explain Analyze on Query 1

```
| -> Table scan on <temporary>  (actual time=2.473..2.476 rows=20 loops=1)
    -> Aggregate using temporary table  (actual time=2.472..2.472 rows=20 loops=1)
        -> Table scan on SubQuery  (cost=2.50..2.50 rows=0) (actual time=2.344..2.351 rows=50 loops=1)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=2.343..2.343 rows=50 loops=1)
                -> Table scan on <temporary>  (actual time=2.294..2.303 rows=50 loops=1)
                    -> Aggregate using temporary table  (actual time=2.292..2.292 rows=50 loops=1)
                        -> Nested loop inner join  (cost=909.29 rows=140) (actual time=0.331..1.961 rows=220 loops=1)
                            -> Nested loop inner join  (cost=860.24 rows=140) (actual time=0.323..1.840 rows=220
loops=1)
                                -> Nested loop inner join  (cost=291.10 rows=146) (actual time=0.274..1.135 rows=20
loops=1)
                                    -> Covering index scan on Course using faculty_idx  (cost=21.00 rows=200) (actual
time=0.040..0.082 rows=200 loops=1)
                                    -> Filter: (Assignment.AssignmentID like '%_9D_%')  (cost=0.69 rows=1) (actual
time=0.005..0.005 rows=0 loops=200)
                                        -> Covering index lookup on Assignment using course_idx
(CourseID=Course.CourseID)  (cost=0.69 rows=7) (actual time=0.003..0.004 rows=5 loops=200)
                                -> Filter: (Question.`Type` = 'O')  (cost=2.94 rows=1) (actual time=0.014..0.032 rows=11
loops=20)
                                    -> Index lookup on Question using AssignmentID
(AssignmentID=Assignment.AssignmentID)  (cost=2.94 rows=10) (actual time=0.014..0.030 rows=11 loops=20)
                            -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)
(cost=0.25 rows=1) (actual time=0.000..0.000 rows=1 loops=220)
 |
```

2) Add Index On AssumedDifficulty

```
mysql>
mysql>
mysql> SHOW INDEXES FROM Question;
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+--------------
| Table    | Non_unique | Key_name               | Seq_in_index | Column_name       | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expressi
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+---------
| Question |          0 | PRIMARY                |            1 | QuestionID        | A         |        9547 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
| Question |          1 | AssignmentID           |            1 | AssignmentID      | A         |         993 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
| Question |          1 | AssumedDifficulty_index |           1 | AssumedDifficulty | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+---------
3 rows in set (0.06 sec)

mysql>
```

Fig: Index AssumedDifficulty_index added to Question(AssumedDifficulty)

```
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-------------------------------+
| -> Table scan on <temporary>  (actual time=2.439..2.442 rows=20 loops=1)
    -> Aggregate using temporary table  (actual time=2.438..2.438 rows=20 loops=1)
        -> Table scan on SubQuery  (cost=2.50..2.50 rows=0) (actual time=2.311..2.319 rows=50 loops=1)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=2.310..2.310 rows=50 loops=1)
                -> Table scan on <temporary>  (actual time=2.265..2.272 rows=50 loops=1)
                    -> Aggregate using temporary table  (actual time=2.263..2.263 rows=50 loops=1)
                        -> Nested loop inner join  (cost=947.43 rows=140) (actual time=0.384..1.917 rows=220 loops=1)
                            -> Nested loop inner join  (cost=898.38 rows=140) (actual time=0.376..1.812 rows=220 loops=1)
                                -> Nested loop inner join  (cost=291.10 rows=146) (actual time=0.327..1.189 rows=20 loops=1)
                                    -> Covering index scan on Course using faculty_idx  (cost=21.00 rows=200) (actual time=0.090..0.130 rows=200 loops=1)
                                    -> Filter: (Assignment.AssignmentID like '%_9D_%')  (cost=0.69 rows=1) (actual time=0.005..0.005 rows=0 loops=200)
                                        -> Covering index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=0.69 rows=7) (actual time=0.003..0.004 rows=5 loops=200)
                                -> Filter: (Question.`Type` = 'O')  (cost=3.21 rows=1) (actual time=0.014..0.030 rows=11 loops=20)
                                    -> Index lookup on Question using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=3.21 rows=10) (actual time=0.013..0.029 rows=11 loops=20)
                            -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)  (cost=0.25 rows=1) (actual time=0.000..0.000 rows=1 loops=220)
 |
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-------------------------------+
```

Fig: Explain Analyze on Query 1 after adding Index mentioned above

```
| -> Table scan on <temporary>  (actual time=2.439..2.442 rows=20 loops=1)
    -> Aggregate using temporary table  (actual time=2.438..2.438 rows=20 loops=1)
        -> Table scan on SubQuery  (cost=2.50..2.50 rows=0) (actual time=2.311..2.319 rows=50 loops=1)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=2.310..2.310 rows=50 loops=1)
                -> Table scan on <temporary>  (actual time=2.265..2.272 rows=50 loops=1)
                    -> Aggregate using temporary table  (actual time=2.263..2.263 rows=50 loops=1)
                        -> Nested loop inner join  (cost=947.43 rows=140) (actual time=0.384..1.917 rows=220 loops=1)
                            -> Nested loop inner join  (cost=898.38 rows=140) (actual time=0.376..1.812 rows=220
loops=1)
                                -> Nested loop inner join  (cost=291.10 rows=146) (actual time=0.327..1.189 rows=20
loops=1)
                                    -> Covering index scan on Course using faculty_idx  (cost=21.00 rows=200) (actual
time=0.090..0.130 rows=200 loops=1)
                                    -> Filter: (Assignment.AssignmentID like '%_9D_%')  (cost=0.69 rows=1) (actual
time=0.005..0.005 rows=0 loops=200)
                                        -> Covering index lookup on Assignment using course_idx
(CourseID=Course.CourseID)  (cost=0.69 rows=7) (actual time=0.003..0.004 rows=5 loops=200)
                                -> Filter: (Question.`Type` = 'O')  (cost=3.21 rows=1) (actual time=0.014..0.030 rows=11
loops=20)
                                    -> Index lookup on Question using AssignmentID
(AssignmentID=Assignment.AssignmentID)  (cost=3.21 rows=10) (actual time=0.013..0.029 rows=11 loops=20)
                            -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)
(cost=0.25 rows=1) (actual time=0.000..0.000 rows=1 loops=220)
 |
```

<u>CONCLUSION:</u> Cost increasing(with respect to Config 1) after index addition on **AssumedDifficulty**.

3) Add index on AssumedDifficulty and Type

```
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW INDEXES FROM Question;
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table    | Non_unique | Key_name               | Seq_in_index | Column_name       | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Question |          0 | PRIMARY                |            1 | QuestionID        | A         |        9547 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Question |          1 | AssignmentID           |            1 | AssignmentID      | A         |         993 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Question |          1 | AssumedDifficulty_index |           1 | AssumedDifficulty | A         |           3 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Question |          1 | QuestionType_index     |            1 | Type              | A         |           1 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
+----------+------------+------------------------+--------------+-------------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
4 rows in set (0.02 sec)

mysql>
```

Fig: Index AssumedDifficulty_index added to Question(AssumedDifficulty) and QuestionType_index to Question(Type)

```
--------------------------+
| -> Table scan on <temporary>  (actual time=2.421..2.424 rows=20 loops=1)
    -> Aggregate using temporary table  (actual time=2.420..2.420 rows=20 loops=1)
        -> Table scan on SubQuery  (cost=2.50..2.50 rows=0) (actual time=2.293..2.301 rows=50 loops=1)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=2.292..2.292 rows=50 loops=1)
                -> Table scan on <temporary>  (actual time=2.250..2.257 rows=50 loops=1)
                    -> Aggregate using temporary table  (actual time=2.248..2.248 rows=50 loops=1)
                        -> Nested loop inner join  (cost=968.46 rows=1401) (actual time=0.332..1.822 rows=220 loops=1)
                            -> Nested loop inner join  (cost=342.12 rows=146) (actual time=0.284..1.183 rows=20 loops=1)
                                -> Nested loop inner join  (cost=291.10 rows=146) (actual time=0.276..1.159 rows=20 loops=1)
                                    -> Covering index scan on Course using faculty_idx  (cost=21.00 rows=200) (actual time=0.045..0.087 rows=200 loops=1)
                                    -> Filter: (Assignment.AssignmentID like '%_9D_%')  (cost=0.69 rows=1) (actual time=0.005..0.005 rows=0 loops=200)
                                        -> Covering index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=0.69 rows=7) (actual time=0.003..0.004 rows=5 loops=200)
                                -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)  (cost=0.25 rows=1) (actual time=0.001..0.001 rows=1 loops=20)
                            -> Filter: (Question.`Type` = 'O')  (cost=3.34 rows=10) (actual time=0.014..0.031 rows=11 loops=20)
                                -> Index lookup on Question using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=3.34 rows=10) (actual time=0.014..0.029 rows=11 loops=20)
 |
+--------------------------
```

```
                                -> Single-row covering index lookup on Faculty using PRIMARY (FacultyID=Course.FacultyID)  (cost=0.25 rows=1)
(actual time=0.001..0.001 rows=1 loops=20)
                            -> Filter: (Question.`Type` = 'O')  (cost=3.37 rows=10) (actual time=0.015..0.033 rows=11 loops=20)
                                -> Index lookup on Question using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=3.37 rows=10)
(actual time=0.015..0.031 rows=11 loops=20)
 |
```

CONCLUSION: Cost decreasing (with respect to Config 1) with index addition on Type and AssumedDifficulty.

JUSTIFICATION: We have chosen the configuration that uses Type and AssumedDifficulty as an index. This configuration gave us the lowest cost. This configuration also does not significantly affect the cost of other queries. We noticed that the costs for the configuration with just the type index is very similar to the one we have chosen, but since our application is read-heavy, it would benefit from a higher amount of indexing. Further, the cost of joins has also been reduced.

B) QUERY 2

1) Original Explain Analyze:- without any added indices

```
mysql>
mysql> SHOW INDEXES FROM Student;
+---------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Student |          0 | PRIMARY  |            1 | StudentID   | A         |        1469 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Student |          1 | ClassID  |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.01 sec)

mysql> SHOW INDEXES FROM Class;
+-------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+-------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Class |          0 | PRIMARY  |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+-------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
1 row in set (0.00 sec)

mysql> SHOW INDEXES FROM Course;
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name    | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Course |          0 | PRIMARY     |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | ClassID     |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_idx |            1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.01 sec)

mysql> SHOW INDEXES FROM Assignment;
+------------+------------+------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name   | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY    |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.00 sec)

mysql>
```

Fig: Initial Indexes

```
------------------------+
| -> Table scan on <except temporary>  (cost=40.65..42.96 rows=9) (actual time=0.184..0.185 rows=3 loops=1)
    -> Except materialize with deduplication  (cost=40.36..40.36 rows=9) (actual time=0.183..0.183 rows=4 loops=1)
        -> Table scan on Query1  (cost=31.46..33.76 rows=9) (actual time=0.134..0.135 rows=4 loops=1)
            -> Materialize  (cost=31.16..31.16 rows=9) (actual time=0.133..0.133 rows=4 loops=1)
                -> Table scan on <temporary>  (cost=27.99..30.29 rows=9) (actual time=0.124..0.125 rows=4 loops=1)
                    -> Temporary table with deduplication  (cost=27.69..27.69 rows=9) (actual time=0.123..0.123 rows=4 loops=1)
                        -> Nested loop inner join  (cost=26.81 rows=9) (actual time=0.051..0.108 rows=4 loops=1)
                            -> Covering index lookup on Course using ClassID (ClassID='10A')  (cost=0.67 rows=4) (actual time=0.010..0.011 rows=4 loops=1)
                            -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=5.94 rows=2) (actual time=0.022..0.024 rows=1 loops=4)
                                -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=5.94 rows=7) (actual time=0.021..0.023 rows=5 loops=4)
        -> Table scan on <temporary>  (cost=5.72..5.72 rows=1) (actual time=0.034..0.034 rows=1 loops=1)
            -> Temporary table with deduplication  (cost=3.22..3.22 rows=1) (actual time=0.034..0.034 rows=1 loops=1)
                -> Nested loop inner join  (cost=3.15 rows=1) (actual time=0.021..0.028 rows=1 loops=1)
                    -> Nested loop inner join  (cost=1.16 rows=2) (actual time=0.015..0.019 rows=2 loops=1)
                        -> Covering index lookup on Submits using StudentID (StudentID='SID20131143')  (cost=0.46 rows=2) (actual time=0.005..0.006 rows=2 loops=1)
                        -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.30 rows=1) (actual time=0.006..0.006 rows=1 loops=2)
                    -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=0.91 rows=0.3) (actual time=0.004..0.004 rows=0 loops=2)
                        -> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.91 rows=1) (actual time=0.004..0.004 rows=1 loops=2)
 |
+-----------------------------
```

-> Covering index lookup on Course using ClassID (ClassID='10A') (cost=0.67 rows=4) (actual time=0.010..0.011 rows=4 loops=1)
-> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00') (cost=5.94 rows=2) (actual time=0.022..0.024 rows=1 loops=4)
-> Index lookup on Assignment using course_idx (CourseID=Course.CourseID) (cost=5.94 rows=7) (actual time=0.021..0.023 rows=5 loops=4)
-> Table scan on <temporary> (cost=5.72..5.72 rows=1) (actual time=0.034..0.034 rows=1 loops=1)
-> Temporary table with deduplication (cost=3.22..3.22 rows=1) (actual time=0.034..0.034 rows=1 loops=1)
-> Nested loop inner join (cost=3.15 rows=1) (actual time=0.021..0.028 rows=1 loops=1)
-> Nested loop inner join (cost=1.16 rows=2) (actual time=0.015..0.019 rows=2 loops=1)
-> Covering index lookup on Submits using StudentID (StudentID='SID20131143') (cost=0.46 rows=2) (actual time=0.005..0.006 rows=2 loops=1)
-> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID) (cost=0.30 rows=1) (actual time=0.006..0.006 rows=1 loops=2)
-> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00') (cost=0.91 rows=0.3) (actual time=0.004..0.004 rows=0 loops=2)
-> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID) (cost=0.91 rows=1) (actual time=0.004..0.004 rows=1 loops=2)
|

2) Add Index On Deadline

```
mysql>
mysql>
mysql>
mysql>
mysql> CREATE INDEX Deadline_index ON Assignment(Deadline);
Query OK, 0 rows affected (0.33 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> SHOW INDEXES FROM Assignment;
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name       | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY        |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx     |            1 | CourseID     | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | Deadline_index |            1 | Deadline     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.03 sec)

mysql>
```

Fig: Index Deadline_index added to Assignment(Deadline)

```
----------------------------+
| -> Table scan on <except temporary>  (cost=21.17..23.09 rows=4) (actual time=0.251..0.252 rows=3 loops=1)
    -> Except materialize with deduplication  (cost=20.54..20.54 rows=4) (actual time=0.251..0.251 rows=4 loops=1)
        -> Table scan on Query1  (cost=13.84..15.75 rows=4) (actual time=0.204..0.204 rows=4 loops=1)
            -> Materialize  (cost=13.20..13.20 rows=4) (actual time=0.203..0.203 rows=4 loops=1)
                -> Table scan on <temporary>  (cost=10.89..12.80 rows=4) (actual time=0.195..0.196 rows=4 loops=1)
                    -> Temporary table with deduplication  (cost=10.25..10.25 rows=4) (actual time=0.193..0.193 rows=4 loops=1)
                        -> Nested loop inner join  (cost=9.85 rows=4) (actual time=0.123..0.179 rows=4 loops=1)
                            -> Covering index lookup on Course using ClassID (ClassID='10A')  (cost=0.67 rows=4) (actual time=0.008..0.009 rows=4 loops=1)
                            -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=1.66 rows=1) (actual time=0.040..0.042 rows=1 loops=4)
                                -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.66 rows=7) (actual time=0.039..0.041 rows=5 loops=4)
        -> Table scan on <temporary>  (cost=4.39..4.39 rows=0.3) (actual time=0.033..0.033 rows=1 loops=1)
            -> Temporary table with deduplication  (cost=1.89..1.89 rows=0.3) (actual time=0.033..0.033 rows=1 loops=1)
                -> Nested loop inner join  (cost=1.86 rows=0.3) (actual time=0.019..0.026 rows=1 loops=1)
                    -> Nested loop inner join  (cost=1.16 rows=2) (actual time=0.014..0.019 rows=2 loops=1)
                        -> Covering index lookup on Submits using StudentID (StudentID='SID20131143')  (cost=0.46 rows=2) (actual time=0.006..0.007 rows=2 loops=1)
                        -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.30 rows=1) (actual time=0.005..0.005 rows=1 loops=2)
                    -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=0.26 rows=0.2) (actual time=0.003..0.003 rows=0 loops=2)
                        -> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=2)
 |
+-----------------------------
```

-> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID) (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=2)
|

CONCLUSION: Cost decreasing (with respect to Config 1) with index addition on Deadline.

3) Add index on ClassID in Student

```
mysql>
mysql>
mysql> SHOW INDEXES FROM Student;
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Student |          0 | PRIMARY        |            1 | StudentID   | A         |        1469 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Student |          1 | class_id_index |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.05 sec)

mysql>
```

Fig: Index class_id_index added to Student(ClassID)

```
-----------------------
```

4) Add index on ClassID in Student and Deadline in Assignment

```
mysql> SHOW INDEXES ON Assignment;
ERROR 1064 (42000): You have an error in your SQL syntax; check the manual that corresponds to your MySQL server version for the right syntax to use near 'ON Assignment' at line 1
mysql> SHOW INDEXES FROM Assignment;
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-------
| Table      | Non_unique | Key_name       | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expres
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-------
| Assignment |          0 | PRIMARY        |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
| Assignment |          1 | course_idx     |            1 | CourseID     | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
| Assignment |          1 | deadline_index |            1 | Deadline     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-------
3 rows in set (0.02 sec)

mysql> SHOW INDEXES FROM Student;
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-----------
| Table   | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-----------
| Student |          0 | PRIMARY        |            1 | StudentID   | A         |        1469 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
| Student |          1 | class_id_index |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+-----------
2 rows in set (0.00 sec)

mysql>
```

Fig: Index Deadline_index added to Assignment(Deadline) and class_id_index added to Student(ClassID)

```
-----------------------------+
| -> Table scan on <except temporary>  (cost=21.17..23.09 rows=4) (actual time=0.188..0.189 rows=3 loops=1)
    -> Except materialize with deduplication  (cost=20.54..20.54 rows=4) (actual time=0.188..0.188 rows=4 loops=1)
        -> Table scan on Query1  (cost=13.84..15.75 rows=4) (actual time=0.137..0.138 rows=4 loops=1)
            -> Materialize  (cost=13.20..13.20 rows=4) (actual time=0.136..0.136 rows=4 loops=1)
                -> Table scan on <temporary>  (cost=10.89..12.80 rows=4) (actual time=0.127..0.129 rows=4 loops=1)
                    -> Temporary table with deduplication  (cost=10.25..10.25 rows=4) (actual time=0.126..0.126 rows=4 loops=1)
                        -> Nested loop inner join  (cost=9.85 rows=4) (actual time=0.037..0.101 rows=4 loops=1)
                            -> Covering index lookup on Course using ClassID (ClassID='10A')  (cost=0.67 rows=4) (actual time=0.010..0.012 rows=4 loops=1)
                            -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=1.66 rows=1) (actual time=0.019..0.022 rows=1 loops=4)
                                -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.66 rows=7) (actual time=0.018..0.021 rows=5 loops=4)
        -> Table scan on <temporary>  (cost=4.39..4.39 rows=0.3) (actual time=0.036..0.036 rows=1 loops=1)
            -> Temporary table with deduplication  (cost=1.89..1.89 rows=0.3) (actual time=0.035..0.035 rows=1 loops=1)
                -> Nested loop inner join  (cost=1.86 rows=0.3) (actual time=0.021..0.029 rows=1 loops=1)
                    -> Nested loop inner join  (cost=1.16 rows=2) (actual time=0.016..0.021 rows=2 loops=1)
                        -> Covering index lookup on Submits using StudentID (StudentID='SID20131143')  (cost=0.46 rows=2) (actual time=0.007..0.008 rows=2 loops=1)
                        -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.30 rows=1) (actual time=0.005..0.005 rows=1 loops=2)
                    -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=0.26 rows=0.2) (actual time=0.004..0.004 rows=0 loops=2)
                        -> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=2)
 |
+-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------
```

Fig: Explain Analyze on Query 2 after adding Indexes mentioned above

```
| -> Table scan on <except temporary>  (cost=21.17..23.09 rows=4) (actual time=0.188..0.189 rows=3 loops=1)
    -> Except materialize with deduplication  (cost=20.54..20.54 rows=4) (actual time=0.188..0.188 rows=4 loops=1)
        -> Table scan on Query1  (cost=13.84..15.75 rows=4) (actual time=0.137..0.138 rows=4 loops=1)
            -> Materialize  (cost=13.20..13.20 rows=4) (actual time=0.136..0.136 rows=4 loops=1)
                -> Table scan on <temporary>  (cost=10.89..12.80 rows=4) (actual time=0.127..0.129 rows=4 loops=1)
                    -> Temporary table with deduplication  (cost=10.25..10.25 rows=4) (actual time=0.126..0.126 rows=4 loops=1)
                        -> Nested loop inner join  (cost=9.85 rows=4) (actual time=0.037..0.101 rows=4 loops=1)
                            -> Covering index lookup on Course using ClassID (ClassID='10A')  (cost=0.67 rows=4) (actual time=0.010..0.012 rows=4 loops=1)
                            -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=1.66 rows=1) (actual time=0.019..0.022 rows=1 loops=4)
                                -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.66 rows=7) (actual time=0.018..0.021 rows=5 loops=4)
        -> Table scan on <temporary>  (cost=4.39..4.39 rows=0.3) (actual time=0.036..0.036 rows=1 loops=1)
            -> Temporary table with deduplication  (cost=1.89..1.89 rows=0.3) (actual time=0.035..0.035 rows=1 loops=1)
                -> Nested loop inner join  (cost=1.86 rows=0.3) (actual time=0.021..0.029 rows=1 loops=1)
                    -> Nested loop inner join  (cost=1.16 rows=2) (actual time=0.016..0.021 rows=2 loops=1)
                        -> Covering index lookup on Submits using StudentID (StudentID='SID20131143')  (cost=0.46 rows=2) (actual time=0.007..0.008 rows=2 loops=1)
                        -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.30 rows=1) (actual time=0.005..0.005 rows=1
loops=2)
                    -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-01 00:00:00')  (cost=0.26 rows=0.2) (actual time=0.004..0.004 rows=0 loops=2)
                        -> Single-row index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1
loops=2)
 |
```

<u>CONCLUSION:</u> Cost decreasing (with respect to Config 1) with index addition on Deadline and Student.ClassID.

<u>JUSTIFICATION:</u> We have chosen the configuration that uses ‘ClassID’ and ‘Deadline’ as an index. While this configuration has the same cost as an index on only ‘Deadline’, we are going with the higher number of indexes due to the focus on read operations in our application, which will involve ClassID and Deadline. This configuration also does not significantly affect the cost of other queries.

C) <u>QUERY 3</u>

1) Original Explain Analyze:- without any added indices

```
mysql> show indexes from Student;
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name       | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Student |          0 | PRIMARY        |            1 | StudentID   | A         |        1469 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Student |          1 | class_id_index |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+----------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.01 sec)

mysql> show indexes from Submits;
+--------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submits |          0 | PRIMARY  |            1 | SubmissionID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submits |          0 | PRIMARY  |            2 | StudentID   | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submits |          1 | StudentID |            1 | StudentID   | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.00 sec)

mysql> show indexes from Submission;
+------------+------------+-----------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name                    | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+-----------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submission |          0 | PRIMARY                     |            1 | SubmissionID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | AssignmentID                |            1 | AssignmentID | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | Submission_faculty_id_index |            1 | FacultyID   | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+-----------------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.01 sec)

mysql> show indexes from Assignment;
+------------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY  |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.00 sec)
```

Fig: Initial Indexes

```
-> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.169..0.400 rows=5 loops=1)
  -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.047..0.281 rows=12 loops=1)
      -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.039..0.238 rows=12 loops=1)
          -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.020..0.023 rows=12 loops=1)
          -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual time=0.018..0.018 rows=1 loops=12)
      -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=12)
  -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.009..0.009 rows=0 loops=12)
      -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.009..0.009 rows=1 loops=12)
          -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.093..0.093 rows=3 loops=1)
              -> Table scan on <temporary>  (actual time=0.070..0.070 rows=3 loops=1)
                  -> Aggregate using temporary table  (actual time=0.069..0.069 rows=3 loops=1)
                      -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.024..0.039 rows=12 loops=1)
                          -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.012..0.016 rows=12 loops=1)
                          -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
|
```

Fig: Initial Explain Analyze on Query 3

```
-> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.169..0.400 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.047..0.281 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.039..0.238 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.020..0.023 rows=12
loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26
rows=1) (actual time=0.018..0.018 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual
time=0.003..0.003 rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.009..0.009 rows=0
loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual
time=0.009..0.009 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.093..0.093 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.070..0.070 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.069..0.069 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.024..0.039 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.012..0.016 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY
(AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
 |
```

2) Add Index ON TotalScore

```
mysql> create index total_score_index on Submission(TotalScore);
Query OK, 0 rows affected (0.14 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show indexes from Submission;
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name                   | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submission |          0 | PRIMARY                    |            1 | SubmissionID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | AssignmentID               |            1 | AssignmentID | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | Submission_faculty_id_index |            1 | FacultyID    | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | total_score_index          |            1 | TotalScore   | A         |          11 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
4 rows in set (0.01 sec)
```

Fig: Index total_score_index added to Submission(TotalScore) and class_id_index added to Student(ClassID)

```
| -> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.147..0.196 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.034..0.084 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.028..0.057 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.013..0.016 rows=12 loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.009..0.009 rows=0 loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.008..0.008 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.086..0.086 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.069..0.070 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.068..0.068 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.032..0.046 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.015..0.018 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
 |
```

Fig: Explain Analyze on Query 3 after adding Index mentioned above


```
-> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.147..0.196 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.034..0.084 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.028..0.057 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.013..0.016 rows=12
loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26
rows=1) (actual time=0.003..0.003 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual
time=0.002..0.002 rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.009..0.009 rows=0
loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual
time=0.008..0.008 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.086..0.086 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.069..0.070 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.068..0.068 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.032..0.046 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.015..0.018 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY
(AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
 |
```


<u>CONCLUSION:</u> Cost unchanged (with respect to Config 1) with index addition on TotalScore.

3) Add index on deadline and TotalScore

```
mysql> show indexes from Assignment;
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name       | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY        |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx     |            1 | CourseID     | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | deadline_index |            1 | Deadline     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.01 sec)

mysql>
```


```
mysql> create index total_score_index on Submission(TotalScore);
Query OK, 0 rows affected (0.14 sec)
Records: 0  Duplicates: 0  Warnings: 0

mysql> show indexes from Submission;
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name                   | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submission |          0 | PRIMARY                    |            1 | SubmissionID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | AssignmentID               |            1 | AssignmentID | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | Submission_faculty_id_index |            1 | FacultyID    | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | total_score_index          |            1 | TotalScore   | A         |          11 |     NULL |   NULL | YES  | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
4 rows in set (0.01 sec)
```


Fig: Index total_score_index added to Submission(TotalScore) and deadline_index added to Assignment(Deadline)

```
| -> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.149..0.197 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.039..0.089 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.032..0.061 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.015..0.018 rows=12 loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.008..0.009 rows=0 loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.008..0.008 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.083..0.083 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.064..0.065 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.064..0.064 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.026..0.041 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.015..0.018 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
```

Fig: Explain Analyze on Query 3 after adding Indexes mentioned above


```
| -> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.149..0.197 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.039..0.089 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.032..0.061 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.015..0.018 rows=12 loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual
time=0.003..0.003 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.002..0.002
rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.008..0.009 rows=0 loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.008..0.008 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.083..0.083 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.064..0.065 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.064..0.064 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.026..0.041 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.015..0.018 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)
(cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
 |
```


<u>CONCLUSION:</u> Cost unchanged (with respect to Config 1) with index addition on TotalScore and Deadline.

4) Add index on deadline

```
mysql> show indexes from Assignment;
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name       | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY        |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx     |            1 | CourseID     | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | deadline_index |            1 | Deadline     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.01 sec)

mysql>
```

Fig: Index deadline_index added to Assignment(Deadline)

```
| -> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.201..0.278 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.047..0.125 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.038..0.084 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.019..0.023 rows=12 loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual time=0.005..0.005 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.003..0.003 rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.012..0.012 rows=0 loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.011..0.012 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.115..0.115 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.092..0.092 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.091..0.091 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.036..0.059 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.024..0.029 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)  (cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
```

Fig: Explain Analyze on Query 3 after adding Index mentioned above


```
| -> Nested loop inner join  (cost=17.50 rows=7) (actual time=0.201..0.278 rows=5 loops=1)
    -> Nested loop inner join  (cost=9.80 rows=11) (actual time=0.047..0.125 rows=12 loops=1)
        -> Nested loop inner join  (cost=5.95 rows=11) (actual time=0.038..0.084 rows=12 loops=1)
            -> Covering index scan on Submits using StudentID  (cost=2.10 rows=11) (actual time=0.019..0.023 rows=12 loops=1)
            -> Single-row index lookup on Submission using PRIMARY (SubmissionID=Submits.SubmissionID)  (cost=0.26 rows=1) (actual
time=0.005..0.005 rows=1 loops=12)
        -> Single-row index lookup on Student using PRIMARY (StudentID=Submits.StudentID)  (cost=0.26 rows=1) (actual time=0.003..0.003
rows=1 loops=12)
    -> Filter: (Submission.TotalScore < T1.avg_score)  (cost=0.51..0.51 rows=1) (actual time=0.012..0.012 rows=0 loops=12)
        -> Covering index lookup on T1 using <auto_key0> (AssignmentID=Submission.AssignmentID)  (actual time=0.011..0.012 rows=1 loops=12)
            -> Materialize  (cost=0.00..0.00 rows=0) (actual time=0.115..0.115 rows=3 loops=1)
                -> Table scan on <temporary>  (actual time=0.092..0.092 rows=3 loops=1)
                    -> Aggregate using temporary table  (actual time=0.091..0.091 rows=3 loops=1)
                        -> Nested loop inner join  (cost=5.65 rows=12) (actual time=0.036..0.059 rows=12 loops=1)
                            -> Table scan on Submission  (cost=1.45 rows=12) (actual time=0.024..0.029 rows=12 loops=1)
                            -> Single-row covering index lookup on Assignment using PRIMARY (AssignmentID=Submission.AssignmentID)
(cost=0.26 rows=1) (actual time=0.002..0.002 rows=1 loops=12)
 |
```

CONCLUSION: Cost unchanged (with respect to Config 1) with index addition on Deadline.

JUSTIFICATION: For this query, we decided to go with the default indexing i.e. the primary and foreign keys since we saw no changes in cost with the relevant configurations that we tested. However, the column 'deadline' is indexed due to our results from a previous query. Hence the addition of TotalScore as an index seemed unnecessary. The table Submission is also going to include a lot of write operations which benefits from lesser indexing.

D) QUERY 4

1) Original Explain Analyze:- without any added indices

```
mysql> show indexes from Course;
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name    | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Course |          0 | PRIMARY     |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | ClassID     |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_idx |            1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+-------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.00 sec)

mysql> show indexes from Assignment;
+------------+------------+-------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name    | Seq_in_index | Column_name   | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+-------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY     |            1 | AssignmentID  | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx  |            1 | CourseID      | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+-------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
2 rows in set (0.00 sec)

mysql> show indexes from Submission;
+------------+------------+-----------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name                    | Seq_in_index | Column_name   | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+-----------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submission |          0 | PRIMARY                     |            1 | SubmissionID  | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | AssignmentID                |            1 | AssignmentID  | A         |           2 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submission |          1 | Submission_faculty_id_index |            1 | FacultyID     | A         |           1 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+-----------------------------+--------------+---------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.00 sec)

mysql> show indexes from Submits;
+---------+------------+-----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table   | Non_unique | Key_name  | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+---------+------------+-----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Submits |          0 | PRIMARY   |            1 | SubmissionID | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submits |          0 | PRIMARY   |            2 | StudentID   | A         |          11 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Submits |          1 | StudentID |            1 | StudentID   | A         |          10 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+---------+------------+-----------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.00 sec)
```

Fig: Initial Indexes

```
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=54.60 rows=66) (actual time=0.216..0.334 rows=5 loops=1)
    -> Nested loop left join  (cost=48.04 rows=66) (actual time=0.082..0.296 rows=19 loops=1)
        -> Nested loop left join  (cost=25.09 rows=66) (actual time=0.073..0.245 rows=19 loops=1)
            -> Nested loop inner join  (cost=13.06 rows=11) (actual time=0.051..0.171 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_idx (FacultyID='IU175125')  (cost=1.58 rows=5) (actual time=0.013..0.015 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.028..0.030 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.027..0.029 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.55 rows=6) (actual time=0.006..0.007 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual time=0.002..0.002 rows=1 loops=19)
 |
```

Fig:Initial Explain Analyze on Query 4

```
 -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=54.60 rows=66) (actual time=0.216..0.334 rows=5 loops=1)
    -> Nested loop left join  (cost=48.04 rows=66) (actual time=0.082..0.296 rows=19 loops=1)
        -> Nested loop left join  (cost=25.09 rows=66) (actual time=0.073..0.245 rows=19 loops=1)
            -> Nested loop inner join  (cost=13.06 rows=11) (actual time=0.051..0.171 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_idx (FacultyID='IU175125')  (cost=1.58 rows=5) (actual time=0.013..0.015 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.028..0.030 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.027..0.029 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.55 rows=6) (actual time=0.006..0.007 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual time=0.002..0.002 rows=1 loops=19)
 |
```

2) Add Index ON deadline

```
mysql> show indexes from Assignment;
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table      | Non_unique | Key_name       | Seq_in_index | Column_name  | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Assignment |          0 | PRIMARY        |            1 | AssignmentID | A         |        1312 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | course_idx     |            1 | CourseID     | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Assignment |          1 | deadline_index |            1 | Deadline     | A         |           5 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+------------+------------+----------------+--------------+--------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
3 rows in set (0.01 sec)
```

Fig: Index deadline_index added to Assignment(Deadline) and class_id_index added to Student(ClassID)

```
-----------------------------------------------------+
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=51.06 rows=60) (actual time=0.250..0.319 rows=5 loops=1)
    -> Nested loop left join  (cost=45.06 rows=60) (actual time=0.102..0.287 rows=19 loops=1)
        -> Nested loop left join  (cost=24.06 rows=60) (actual time=0.094..0.226 rows=19 loops=1)
            -> Nested loop inner join  (cost=13.06 rows=10) (actual time=0.046..0.109 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_idx (FacultyID='IU175125')  (cost=1.58 rows=5) (actual time=0.012..0.015 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.016..0.018 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.015..0.017 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.56 rows=6) (actual time=0.008..0.011 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual time=0.002..0.003 rows=1 loops=19)
 |
```

Fig: Explain Analyze on Query 4 after adding Index mentioned above

```
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=51.06 rows=60) (actual
time=0.250..0.319 rows=5 loops=1)
    -> Nested loop left join  (cost=45.06 rows=60) (actual time=0.102..0.287 rows=19 loops=1)
        -> Nested loop left join  (cost=24.06 rows=60) (actual time=0.094..0.226 rows=19 loops=1)
            -> Nested loop inner join  (cost=13.06 rows=10) (actual time=0.046..0.109 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_idx (FacultyID='IU175125')  (cost=1.58 rows=5) (actual
time=0.012..0.015 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual
time=0.016..0.018 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7)
(actual time=0.015..0.017 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.56 rows=6)
(actual time=0.008..0.011 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1)
(actual time=0.002..0.003 rows=1 loops=19)
 |
```

<u>CONCLUSION:</u> Cost decreased (with respect to Config 1) with index addition on Deadline.

3) Add index on deadline and Course.FacultyID

```
mysql> show indexes from Course;
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Course |          0 | PRIMARY          |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | ClassID          |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_idx      |            1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_id__index |           1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
4 rows in set (0.03 sec)
```

Fig: Index faculty_id_index added to Course(FacultyID) and deadline_index added to Assignment(Deadline)

```
-----------------------------------------------------+
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=50.25 rows=60) (actual time=0.180..0.243 rows=5 loops=1)
    -> Nested loop left join  (cost=44.25 rows=60) (actual time=0.090..0.219 rows=19 loops=1)
        -> Nested loop left join  (cost=23.25 rows=60) (actual time=0.081..0.169 rows=19 loops=1)
            -> Nested loop inner join  (cost=12.25 rows=10) (actual time=0.057..0.108 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_id__index (FacultyID='IU175125')  (cost=0.77 rows=5) (actual time=0.020..0.021 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.015..0.017 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.015..0.016 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.56 rows=6) (actual time=0.005..0.006 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual time=0.002..0.002 rows=1 loops=19)
 |
```

Fig: Explain Analyze on Query 4 after adding Indexes mentioned above

```
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=50.25 rows=60) (actual time=0.180..0.243 rows=5
loops=1)
    -> Nested loop left join  (cost=44.25 rows=60) (actual time=0.090..0.219 rows=19 loops=1)
        -> Nested loop left join  (cost=23.25 rows=60) (actual time=0.081..0.169 rows=19 loops=1)
            -> Nested loop inner join  (cost=12.25 rows=10) (actual time=0.057..0.108 rows=10 loops=1)
```

```
                -> Covering index lookup on Course using faculty_id__index (FacultyID='IU175125')  (cost=0.77 rows=5) (actual
time=0.020..0.021 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.015..0.017 rows=2
loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.015..0.016
rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.56 rows=6) (actual
time=0.005..0.006 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual
time=0.002..0.002 rows=1 loops=19)
 |
```

CONCLUSION: Cost decreased (with respect to Config 1) with index addition on Deadline and Course.FacultyID.

4) Add index on Course.FacultyID

```
mysql> show indexes from Course;
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Table  | Non_unique | Key_name         | Seq_in_index | Column_name | Collation | Cardinality | Sub_part | Packed | Null | Index_type | Comment | Index_comment | Visible | Expression |
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
| Course |          0 | PRIMARY          |            1 | CourseID    | A         |         200 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | ClassID          |            1 | ClassID     | A         |          50 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_idx      |            1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
| Course |          1 | faculty_id__index |            1 | FacultyID   | A         |          40 |     NULL |   NULL |      | BTREE      |         |               | YES     | NULL       |
+--------+------------+------------------+--------------+-------------+-----------+-------------+----------+--------+------+------------+---------+---------------+---------+------------+
4 rows in set (0.03 sec)
```

Fig: Index faculty_id_index added to Course(FacultyID)

```
-----------------------------------------------------------+
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=53.79 rows=66) (actual time=0.474..0.595 rows=5 loops=1)
    -> Nested loop left join  (cost=47.23 rows=66) (actual time=0.292..0.548 rows=19 loops=1)
        -> Nested loop left join  (cost=24.28 rows=66) (actual time=0.225..0.422 rows=19 loops=1)
            -> Nested loop inner join  (cost=12.25 rows=11) (actual time=0.180..0.328 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_id__index (FacultyID='IU175125')  (cost=0.77 rows=5) (actual time=0.032..0.034 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.057..0.058 rows=2 loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.056..0.057 rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.55 rows=6) (actual time=0.007..0.009 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual time=0.006..0.006 rows=1 loops=19)
 |
```

Fig: Explain Analyze on Query 4 after adding Index mentioned above

```
| -> Group aggregate: avg(Submission.TotalScore), count(distinct Submits.StudentID)  (cost=53.79 rows=66) (actual time=0.474..0.595 rows=5
loops=1)
    -> Nested loop left join  (cost=47.23 rows=66) (actual time=0.292..0.548 rows=19 loops=1)
        -> Nested loop left join  (cost=24.28 rows=66) (actual time=0.225..0.422 rows=19 loops=1)
            -> Nested loop inner join  (cost=12.25 rows=11) (actual time=0.180..0.328 rows=10 loops=1)
                -> Covering index lookup on Course using faculty_id__index (FacultyID='IU175125')  (cost=0.77 rows=5) (actual
time=0.032..0.034 rows=5 loops=1)
                -> Filter: (Assignment.Deadline < TIMESTAMP'2024-04-28 00:00:00')  (cost=1.68 rows=2) (actual time=0.057..0.058 rows=2
loops=5)
                    -> Index lookup on Assignment using course_idx (CourseID=Course.CourseID)  (cost=1.68 rows=7) (actual time=0.056..0.057
rows=5 loops=5)
            -> Index lookup on Submission using AssignmentID (AssignmentID=Assignment.AssignmentID)  (cost=0.55 rows=6) (actual
time=0.007..0.009 rows=1 loops=10)
        -> Covering index lookup on Submits using PRIMARY (SubmissionID=Submission.SubmissionID)  (cost=0.25 rows=1) (actual
time=0.006..0.006 rows=1 loops=19)
 |
```

CONCLUSION: Cost reduced (with respect to Config 1) with index addition on Course.FacultyID.

JUSTIFICATION: We have chosen the configuration which uses both Course.FacultyID and Deadline as an index. This configuration produces the least cost among other possible index combinations as both these columns are required to apply the WHERE condition in the query. Their combined effect on the cost is greater than the individual impact of each of these column indexes. From the perspective of overall query performance, these indexes do not impact the cost in other queries and result in a minor cost reduction in Query 1 as well.

Optimal Indexing Configuration

The combined indexing results in a significant cost reduction across all 4 queries except for query 3 where the cost remains largely unaffected with a minimal increase.
The optimal indexing structure is:

1) Question.AssumedDifficulty
2) Question.Type
3) Assignment.Deadline
4) Student.ClassID
5) Course.FacultyID